



Operaciones con valores binarios

Introducción al computador y el lenguaje C

Ingeniería de Sistemas y Automática - Universidad de Oviedo

Operaciones con valores binarios

Recordatorio: un entero se codifica en el computador con valores binarios (codificación ponderada en base 2).

```
int x = 23;  
000...010111
```

El nº de bits utilizados depende del tipo de datos y del procesador. Se puede saber con el cálculo `8*sizeof(int)`.

En ciertas ocasiones, el programador puede tener interés en el valor particular de algún(os) bit(s), y no en el valor completo (la cantidad que representa):

- Cuando los bits representan el estado de valores binarios diferentes (ej. E/S digital).
- Cuando se desea comprimir informaciones de varios valores en un solo entero.

Operaciones con valores binarios

En los casos anteriores, el valor decimal y las operaciones aritméticas aportan poca información o es difícil de extraer.

valor = 77; // $2^6 + 2^3 + 2^2 + 2^0$

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	1	0	0	1	1	0	1

- ¿Está activa la luz de peso 2?
- Quiero activar la luz de peso 4.
- Quiero cambiar el estado de las luces de peso 6 y 7 para generar una intermitencia.

Operadores lógicos de bit

Operadores de C para manejar bits (sólo valores enteros):

Binarios (2 operandos):

- **& (AND):** `entA & entB → entR`

Da 1 solo si ambos bits son 1 (ideal para máscaras y aislar bits).

- **| (OR):** `entA | entB → entR`

Da 1 si al menos uno es 1 (ideal para activar bits específicos).

- **^ (XOR):** `entA ^ entB → entR`

Da 1 si los bits son distintos (ideal para invertir o bascular estados).

Monarios (1 operando):

- **~ (NOT):** `~entA → entR`

Invierte cada bit del número sobre el que se aplica.

Ejemplos:

```
int v1 = 77; // 01001101
int v2 = 23; // 00010111

v1 & v2  -> 00000101 (= 5)
v1 | v2  -> 01011111 (= 95)
v1 ^ v2  -> 01011010 (= 90)
~v1      -> 10110010 (= -78)
```

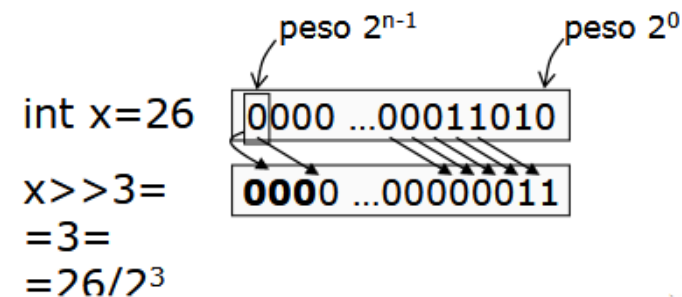
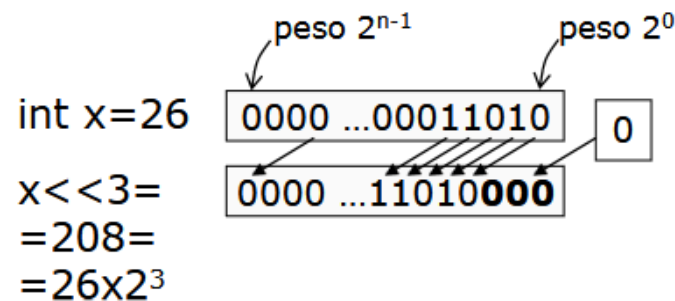
Operadores de desplazamiento de bits

Binarios (2 operandos):

$\ll$ (desplazamiento a izquierda): $\text{entA} \ll \text{entB} \rightarrow \text{entR}$

$\gg$ (desplazamiento a derecha): $\text{entA} \gg \text{entB} \rightarrow \text{entR}$

Nota para $\gg$: Los bits superiores se rellenan con 0 si *entA* es *unsigned*, o con el **bit de signo** de *entA* si es *signed* (0: Positivo, 1: Negativo).



Operaciones con valores binarios (Hexadecimal)

Para indicar constantes de cuyo valor interesan los bits, se usa la codificación hexadecimal (base 16, dígitos 0..9, A..F).

Un código hexadecimal equivale a **4 dígitos binarios**:

0 _h = 0000	1 _h = 0001	2 _h = 0010	3 _h = 0011	4 _h = 0100	5 _h = 0101	6 _h = 0110	7 _h = 0111
8 _h = 1000	9 _h = 1001	A _h = 1010	B _h = 1011	C _h = 1100	D _h = 1101	E _h = 1110	F _h = 1111

En C, las constantes enteras en hexadecimal se preceden de 0x.

```
valor = 77;    // 26 + 23 + 22 + 20 (01001101)
valor = 0x4D; // 4h = 0100, Dh = 1101 (01001101)
```

Se pueden leer/escribir valores en hexadecimal utilizando %x en printf() y scanf() (no usar 0x).

Operaciones con valores binarios

Operaciones habituales con el **bit de peso p** de la variable entera x (afectando sólo a ese bit):

```
int x, p;  
int mascara;  
// ...Dar valores a p y x...  
mascara = 1 << p; // mascara = 0..001 00..000 (2^p)
```

¿Está activo el bit de peso p de x?	<code>if (x & mascara)</code>
Poner a 1 el bit de peso p de x:	<code>x = x mascara;</code>
Poner a 0 el bit de peso p de x:	<code>x = x & ~mascara;</code>
Cambiar el valor del bit de peso p de x:	<code>x = x ^ mascara;</code>

Operaciones con valores binarios

Operaciones habituales con varios datos comprimidos en un entero:

```
int v; // v = ... 25 zzyxxx 20
int vx,vy,vz;

// Extraer vx,vy,vz de v
...Dar valor a v ...
vx= v & 0x7; // vx= ... 000xxx
vy= (v & 0x8) >> 3; // vy= ... 00000y
vz= (v & 0x30) >> 4; // vz= ... 0000zz

// Cambiar valor de los campos de v
...Dar valores a vx,vy,vz ...
v= (v & ~0x7) | (vx & 0x7); // v= ... zzyxxx
v= (v & ~0x8) | ((vy << 3) & 0x8); // v= ... zzyxxx
v= (v & ~0x30) | ((vz << 4) & 0x30); // v= ... zzyxxx
```

Máscara Z → (points to `0x7` and `~0x7`)

Rotación Z → (points to `>> 3` and `<< 4`)

Rotación Z → (points to `<< 3` and `<< 4`)

Máscara Z → (points to `~0x8` and `~0x30`)

Operaciones con valores binarios

Calcular máscara y rotación para un bit de comienzo y un nº de bits de un campo.

- Ej: $v = \dots zzyxxx$ $\text{bitstart_z} = 4, \text{nbits_z} = 2$
 - Rotación = bitstart_z
 - $$\begin{array}{l} \text{2}^n \\ \downarrow \\ \text{▪ } (1 \ll n) = 0001000000 \\ \text{▪ } (1 \ll n) - 1 = 0000111111 \\ \qquad \qquad \qquad \underbrace{\hspace{1.5cm}} \\ \qquad \qquad \qquad \text{n} \end{array}$$
 - $$\begin{array}{l} \text{▪ Máscara} = ((1 \ll (\text{bitstart_z} + \text{nbits_z})) - 1) \ \& \ \sim((1 \ll \text{bitstart_z}) - 1) \\ \qquad \qquad \qquad \underbrace{\hspace{2cm}} \qquad \qquad \qquad \underbrace{\hspace{2cm}} \\ \qquad \qquad \qquad 0 \dots 01111111 \qquad \qquad \qquad 1 \dots 11100000 \\ \qquad \qquad \qquad \underbrace{\hspace{3cm}} \\ \qquad \qquad \qquad 0 \dots 01100000 \end{array}$$
- ```

#define BITSTART_X 0
#define NBITS_X 3
#define MASK_X ((1<<(BITSTART_X+NBITS_X))-1) & ~((1<<BITSTART_X)-1)
#define BITSTART_Y (BITSTART_X+NBITS_X)
#define NBITS_Y 1
#define MASK_Y ((1<<(BITSTART_Y+NBITS_Y))-1) & ~((1<<BITSTART_Y)-1)
#define BITSTART_Z (BITSTART_Y+NBITS_Y)
#define NBITS_Z 2
#define MASK_Z ((1<<(BITSTART_Z+NBITS_Z))-1) & ~((1<<BITSTART_Z)-1)

vz= (v & MASK_Z) >> BITSTART_Z; // vz= ... 0000zz

```

# Ejercicios propuestos

1. Realizar una función que escriba en pantalla una cadena de caracteres que indique el contenido binario de un valor.
2. Realizar una función que pida por teclado una cadena de caracteres que contenga 0s y 1s, y devuelva el valor entero que codifica a dicho valor binario.
3. Usando el simulador (utilizar función `Sleep(200)` para temporizar): Realizar un programa que encienda las luces en función del estado de activación de los interruptores.
4. Realizar un programa que haga parpadear las luces cuyos interruptores están activos.
5. Realizar un programa que permita visualizar una luz que va desplazándose de izquierda a derecha (de forma circular).
6. Realizar un programa que hace parpadear las luces 0 a 3 en función del estado de sus interruptores, y que a la vez traslade una sola luz entre 4 y 7.